

Dzisiaj notatki a takiej pochmurnej szacie graficznej 

Mimo, że na zewnątrz jest upalnie.

Chyba nie muszę tłumaczyć, dlaczego :-)

Używane pojęcia: prefiks, sufix

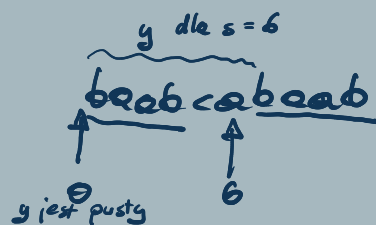
Oznaczenia:  $w \sqsubset x \Leftrightarrow w$  jest prefiksem  $x$

$w \sqsupset x \Leftrightarrow -w-$  sufixem  $x$

Def. Słowo  $P$  występuje w słowie  $T$  z przesunięciem  $s \Leftrightarrow \exists y |y| = s$

oraz  $yP \sqsubset T$ , np. baab występuje z przesunięciem

0 oraz 6 w:



$P$  nazwyjemy wzorcem, a  $T$  nazwyjemy tekstem.

## Problem

Dane:  $P, T$

Zadanie: znaleźć wszystkie wystąpienia  $P$  w  $T$  (tj. podać przesunięcia).

Oznaczenia:  $|T| = n$   $|P| = m$

Jeśli  $X$  to słowo to  $x_1, \dots, x_n$ .

Algorytm naiwny:

Sprawdzamy  $\forall$  przesunięcia od 0 do  $n-m$ .

$T = \underbrace{t_1 t_2 t_3 t_4 t_5 \dots t_n}_{\substack{\uparrow \\ \text{sprawdzamy} \\ \text{czy to} \\ \text{jest wzorcem}}}$

$P = p_1 \dots p_m$

$\underbrace{\hspace{10em}}_{\substack{\uparrow \\ \text{dalej czy to} \\ \text{jest wzorcem}}}$

TC:  $O((n-m+1)m) = O(nm)$

## Co można poprawić?

- wykluczyć bezsensowne przesunięcia
- dla danego przesunięcia przyspieszyć sprawdzenie dopasowania

## Przykład

$P =$  aaaaaaaa

$T =$  aaaaaaa**b**  
nie ma sensu  
sprawdzać tych  
przesunięć

## Algorytm Karpa-Rabina

Niech  $d = |\Sigma|$  (gdzie  $\Sigma$  - alfabet nad którym są  $P, T$ )

Traktujemy  $P_i$  jako liczbę w systemie  $d$ -owym  
(np. dla  $|\Sigma| = 2$  mamy system binarny :-))

$T_i$ :

$T_i$  - liczba  $m$ -cyfrowa odpowiadająca  
fragmentowi tekstu  $t_i \dots t_{i+m-1}$

$P$ :

Obserwacja:  $T_{i+1}$  łatwo obliczyć z  $T_i$ :

$$T_i = t_i \cdot d^{m-1} + \dots + t_{i+m-2} \cdot d^2 + t_{i+m-1} \cdot d + t_{i+m}$$

$$T_{i+1} = t_{i+1} \cdot d^{m-1} + \dots + t_{i+m-1} \cdot d^2 + t_{i+m} \cdot d + t_{i+m+1}$$

$$\text{Stąd } T_{i+1} = (T_i - t_i \cdot d^{m-1}) \cdot d + t_{i+m+1}$$

Jeśli  $t$  jest drugie, to zamiast liczb  $P, T_1, T_2, \dots$   
użyjemy ich haszy (tj. wartości mod  $q$  - pierwsza).

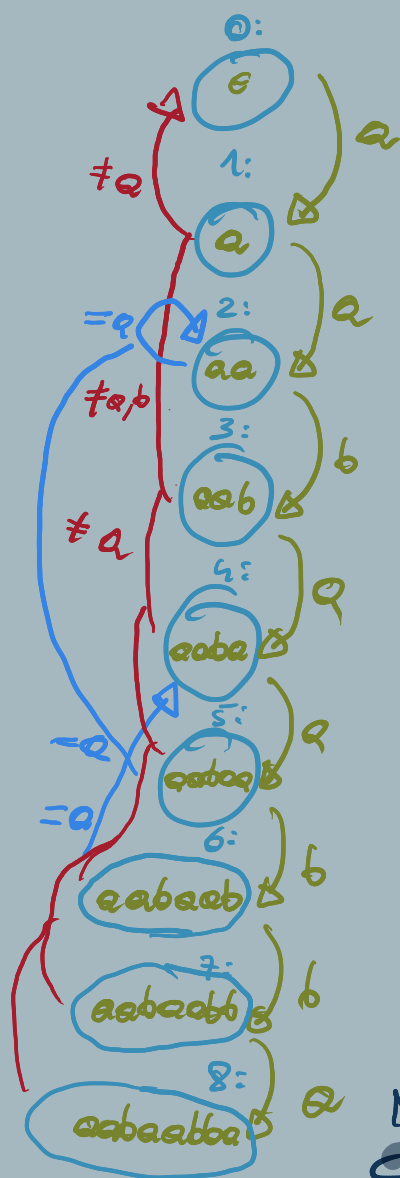
Jeśli  $\text{hasz}(P) \neq \text{hasz}(T_i)$  to wiemy, że wzorec  
nie występuje z przesunięciem  $i-1$   
wpp. sprawdzemy  $P$ ,  $t_i$  literka po literce

## Automat skończony

Automat tworzymy dla wzorca  $P$ , chcemy żeby automat był w stanie  $i$ , kiedy  $i$  jest największe takie, że  $p_1 \dots p_i$  był największym dopasowanym fragmentem

$P = aabaaabba$

$V \rightarrow$  stany (początkowo tyle samo ile liter w  $P + 1$ )



Jak zbudować automat?

Nieumie

for  $q \leftarrow 0$  to  $m$ :  $\leftarrow$  dla każdego stanu  
for each  $a \in \Sigma$ :  $\leftarrow$  dla każdej litery z  $\Sigma$   
     $k \leftarrow \min(m+1, q+2)$   $\leftarrow$  szukamy stanu do którego  
    repeat  $k \leftarrow k-1$  until  $P_k \supset P_q a$   $\leftarrow$  stacamy dla danej litery  
     $\delta(q, a) \leftarrow k$   
return  $\delta$

# Algorytm KMP (Karp, Miller, Rosenberg)

$$\Phi, T \quad w = \Phi T \quad |\Phi| = 2^i$$

Idea - numerujemy w k-tej fazie ( $k=0\dots$ ) podsiłowa w o długości  $2^k$  w porządku leksykograficznym

Przykład

$$\Phi = \text{klpr}$$

$$T = \text{atmbklprkpatklpr}$$

$$\Phi T = \text{klpratmbklprkpatklpr}$$

$$k=0: \text{klpratmbklprkpatklpr}$$

3 4 6 7 1 8 5 2 3 4 6 7 3 6 1 8 3 4 6 4

abklmprt  
1 2 3 4 5 6 7 8

$$k=1: \text{klpratmbklprkpatklpr}$$

kl lp pr ra at mb kl lp pr rk kp pa at tk kl pr  
<3,4> <4,6> <6,7> <7,1> <1,8> <8,5> <5,2> <2,3> <3,4> <4,6> <6,7> <7,1> <1,8> <8,5> <5,2> <2,3>  
3 5 8 9 1 12 6 2 3 5 8 10 4 7 1 11 3 5 8

<1,8> <2,3> <3,4> <3,6> <4,6> <5,2> <6,1> <6,7> <7,1> <7,3> <8,5>  
1 2 3 4 5 6 7 8 9 10 11 12  
sortowanie leksykograficzne

$$k=2: \text{klpratmbklprkpatklpr}$$

```
>>> for i in range(0, slen - 3, 1):
...     print(s[i:i+4])
```

|                |                 |
|----------------|-----------------|
| klpr <3,8>     | atkl <1,3> 1    |
| lp ra <5,9>    | atmb <1,6> 2    |
| pr at <8,1>    | bk lp <2,5> 3   |
| ra tm <9,12>   | kl pr <3,8> 4   |
| at mb <1,6>    | kp at <4,1> 5   |
| tm bk <12,2>   | lp ra <5,9> 6   |
| mb kl <6,3>    | lp rk <5,10> 7  |
| bk lp <2,5>    | mb kl <6,3> 8   |
| kl pr <3,8>    | pa tk <7,10> 9  |
| lp rk <5,10>   | pr at <8,1> 10  |
| lp rk <5,10>   | pr kp <8,4> 11  |
| rk pa <10,4>   | ra tm <9,12> 12 |
| sk pa <4,1>    | rk pa <10,4> 13 |
| g patk <11,11> | tk lp <4,5> 14  |
| at kl <1,3>    | tm bk <12,2> 15 |
| tk lp <11,5>   |                 |
| kl pr <3,8>    |                 |

W momencie, gdy  $2^k = |P|$  sprawdzamy, które przesunięcia miały taki sam numer i zwracamy odpowiedź.

— pozwoliłem sobie rzyć  
REPLa pythona do wypisania  
wszystkich podsiłw 4-literowych.

W przypadku ogólnym  $2^k < |P| < 2^{k+1}$  bierzemy prefiks  $P$  o długości  $2^k$  i sufix o długości  $2^k$ .

Kiedy osiągniemy  $2^k$  dopasujemy prefiks oraz sufix oraz sprawdz, czy odpowiednio się pokrywają  
(tj.  $(\text{pref} \setminus \text{suf}) \cup \text{suf} = P$  i pozycja suf to pozycja pref +  $2^k - 1$ )

Przykład wzorec: klpr kpa klpr atmb klpr kpat klpr  
prefiks  $2^k=4$       sufix  $2^k=4$        $4-4=3$

```
>>> for i in range(0, slen - 3, 1):
...     print(s[i:i+4])
... 
```

|     |      |    |                      |
|-----|------|----|----------------------|
| 1.  | klpr | 5  | klpr (tj. kluc)      |
| 2.  | lprk | 8  | występuje na pozycji |
| 3.  | prkp | 12 | 1, 12, 20            |
| 4.  | rkpa | 13 |                      |
| 5.  | kpa  | 6  | rkpa (tj. 13)        |
| 6.  | paat | 10 | na pozycji           |
| 7.  | aatm | 1  | 4 oraz 15            |
| 8.  | atmb | 3  |                      |
| 9.  | tmbk | 15 |                      |
| 10. | mbkl | 9  | $1+7-4$ $12+7-4$     |
| 11. | bklp | 4  |                      |
| 12. | klpr | 5  | znaleźliśmy wzorec   |
| 13. | lprk | 8  |                      |
| 14. | prkp | 12 |                      |
| 15. | rkpa | 13 |                      |
| 16. | kpat | 7  |                      |
| 17. | patk | 11 |                      |
| 18. | atkl | 2  |                      |
| 19. | tklp | 14 |                      |
| 20. | klpr | 5  |                      |

```
>>> for e in subwords:
...     print(e)
... 
```

|      |    |                      |
|------|----|----------------------|
| aatm | 1  | UWAŁA!               |
| atkl | 2  | 2 racje braku        |
| atmb | 3  | czasu - numerki      |
| bklp | 4  | nadatek tylko        |
| klpr | 5  | na podstawie         |
| kpa  | 6  | sortowania           |
| kpat | 7  | wiec być może        |
| lprk | 8  | algorytm to          |
| mbkl | 9  | zrobi trochę inaczej |
| paat | 10 | Ale to chodzi        |
| patk | 11 | tylko o ileś         |
| prkp | 12 | jest sobie rodzic    |
| rkpa | 13 | gdy $ P  \neq 2^k$   |
| tklp | 14 |                      |
| tmbk | 15 |                      |

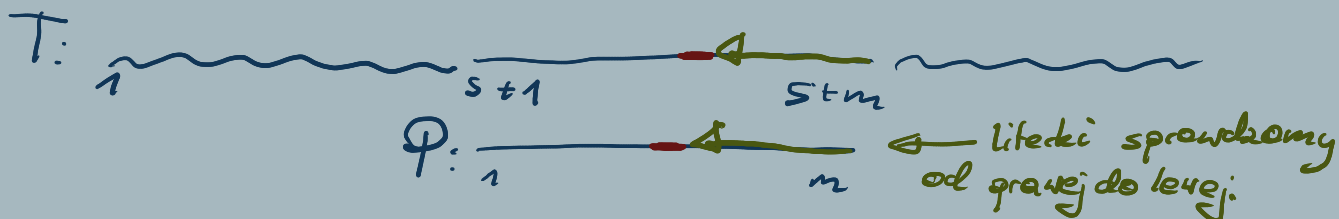
Złożoność  $O(n \log n)$  - ponoc ten algorytm jest dobry w jakiś równoległych wyszczekowaniach.

# Algorytm Boyera-Moore'a

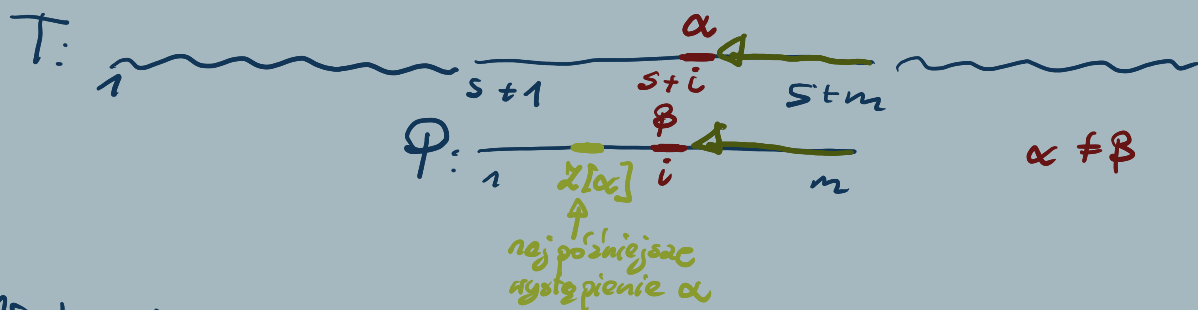
Głównie przez dzień cały kura  
Że z Boylera leci ChMoora

Dla danego przesunięcia azor zgodność sprawdzamy  
z prawa na lewo

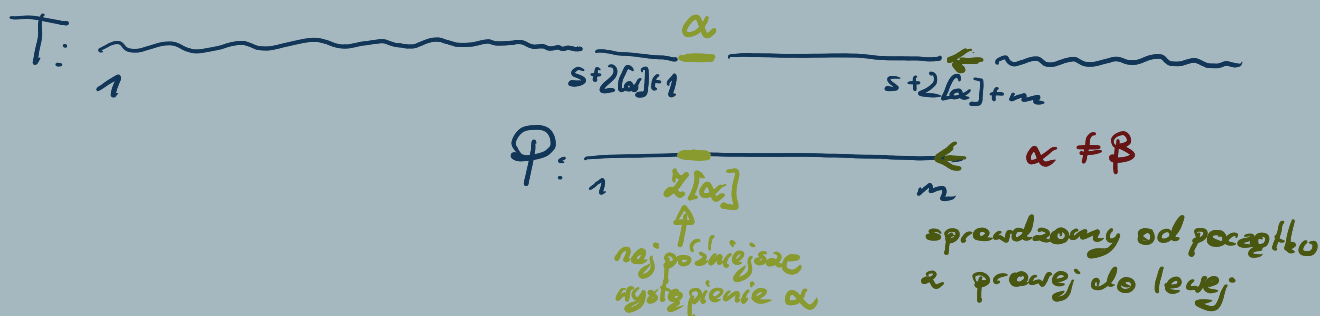
Czyli badając przesunięcie  $s$  mamy:



1<sup>o</sup> cały znak -  $\forall \alpha \in \Sigma$  użyjemy kodicy 2, t.j.  
 $Z[\alpha] = \max \{k, \text{t.j. } P[k] = \alpha\}$



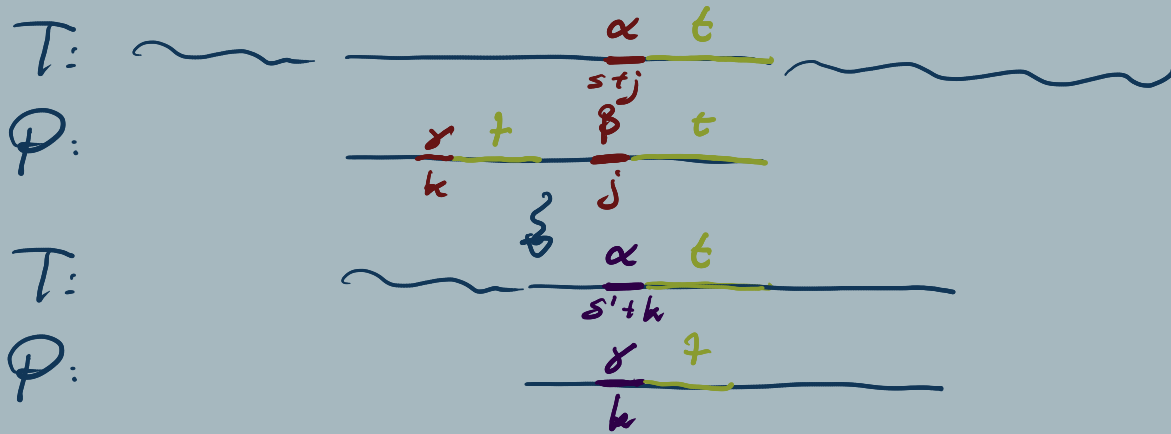
W takiej sytuacji badanie przesunięć dla  $s' = s, s+1, s+[k]-1$   
nie ma sensu, bo na pozycjach  $Z[\alpha]+1 \dots i$  będzie coś innego  
niż  $\alpha$ , więc nic nie dopasujemy - sprawdzamy zatem przesunięcie  
o  $s + Z[\alpha]$



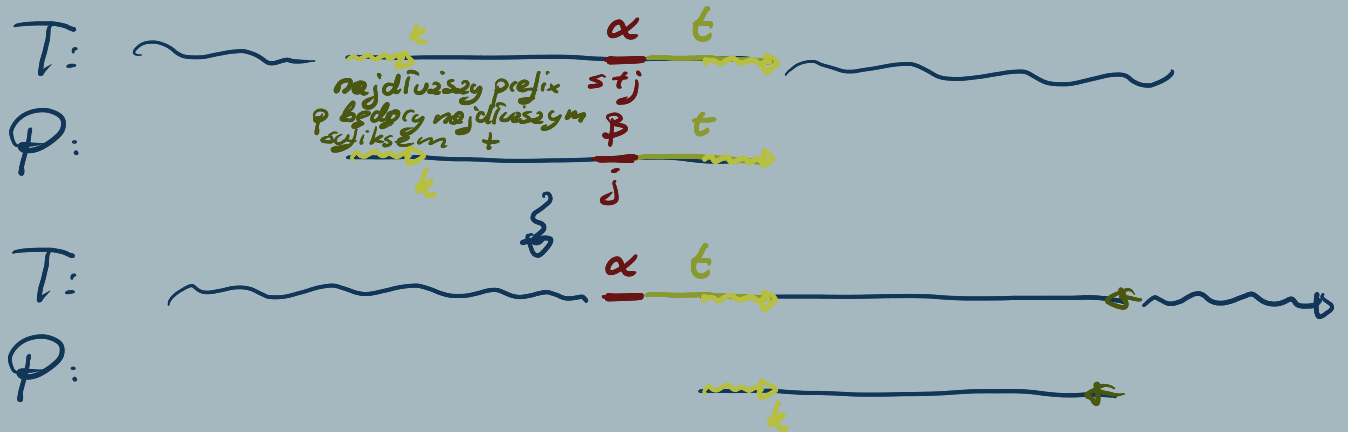
Jeżeli  $\alpha$  nie występuje w wzorcu to sprawdzamy przesunięcie  
 $s+m$

2° 'dobry sufix' -  $S[j] = \max\{k: (\Phi[j+1 \dots m]) \cap \Phi_{k+m-j} \cap \Phi_{k+m-j} \neq \emptyset\}$

to tylko formalna definicja z wykładu, w praktyce chodzi o to, że:



Jeżeli dopasowaliśmy  $t = P[j+1 \dots n]$  i trafiliśmy na mismatch na pozycji  $j$  to szukamy kolejnego najbliżej na prawo (nie będącego sufiksem) wystąpienie  $\tau$



Jeśli takiego nie znaleźliśmy szukamy najbliższego prefiksu  $\Phi$ , który jest sufiksem  $\tau$

Jeśli nic nie ma (ponownie, jak wyżej) przesuwamy  $\Phi$  o  $m$ .

Skomplikowanie:

możemy optymistycznie  $\Theta(\frac{n}{m})$  (gdy  $\Phi \cap T = \emptyset$ ) - skracamy cały tekst o  $m$   
 możemy pesymistycznie  $\Theta(nm)$  - gdy każde przesunięcie jest dopasowaniem, np.  $T = A_{n \text{ razy}}$ ,  $\Phi = A_{m \text{ razy}}$

## Shift-And

$$\Phi = p_1 \dots p_n$$

$\forall j$  chcemy mieć  $C_j$  wektor charakterystyczny dla prefiksów wzorca  
taki, że

$$\text{dla } k=0, \dots, m \quad C_j[k] \equiv \Phi_k \supset T_j$$

$$C_j[k] = \begin{cases} \text{true} & \text{dla } k=0 \\ C_{j-1}[k-1] \wedge p_i = t_j & \text{dla } k>0 \end{cases}$$

Dla każdego  $\delta \in \Sigma$  tworzymy wektor:

$$R_\delta[k] = \begin{cases} \text{true} & \text{iff } p_k = \delta \\ \text{false} & \text{wpp} \end{cases}$$

Wektory reprezentujemy w postaci liczb binarnych.

## Przykład

wzorec: a r a b

$$R_a = \begin{cases} 0101 \\ 0010 \\ 1000 \\ 0000 \end{cases}$$

$$R_r = \begin{cases} 0101 \\ 0010 \\ 1000 \\ 0000 \end{cases}$$

$$R_b = \begin{cases} 0101 \\ 0010 \\ 1000 \\ 0000 \end{cases}$$

$$R_\delta = \begin{cases} 0101 \\ 0010 \\ 1000 \\ 0000 \end{cases} \delta \neq a, r, b$$

$$\text{indeks} \begin{cases} 4 & 3 & 2 & 1 \end{cases}$$

numerujemy od tyłu, jak bity

## Idea algorytmu

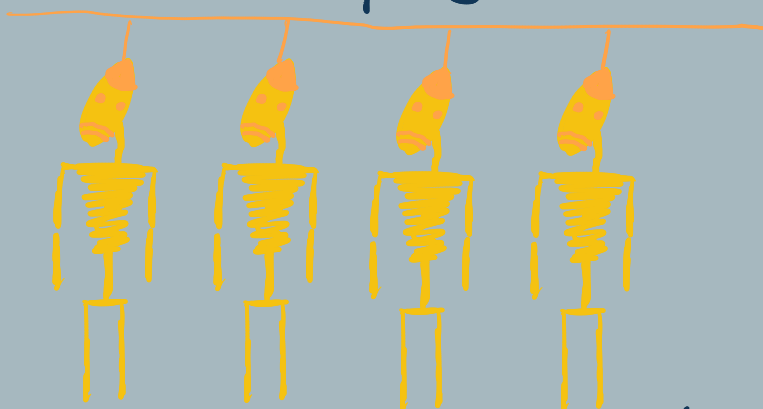
→ Przesuwamy ramkę wielkości wzorca po tekście.

→ Chcemy aby zapalenie  $i$ -tego bitu w tej ramce oznaczało, że dopasowaliśmy tekst to  $i$  pierwszych bitów w danej iteracji

## Pseudokod:

```
frame ← 0
for literka in tekst:
    frame ← frame << 1 (shift u lewo o 1)
    frame ← frame | 1 (zapalamy ostatni bit)
    frame ← frame & R[literka] (and z maską literki)
```

Mnemotechnika z fabryką droidów na **Geonosis**

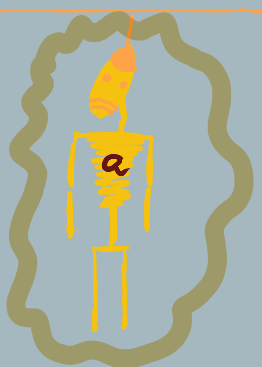


↑  
Planeta ze Star Wars (żeby nie było wątpliwości o pochodzenie)

Fabryka na Geonosis produkuje droidy bojowe (tyle ile wynosi długość tekstu) niestety wąska ramka linii produkcyjnej (długości wzorca)

jest pod ostrzałem armii klonów. Programiści republiki zhotowali ponadto system osłony droidów w konkretnym pobieraniu. Każdy droid ma przypisaną literę i osłona przed ostrzałem droida na i-tej pozycji dzieła wtu litera przypisanemu pierwszemu w rancie (dopiero wyłączonego się) droidowi występuje na i-tej pozycji w wzorcu).

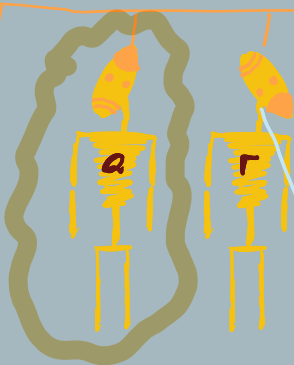
Przykład: Wzorec: arob Słowo: arebarob



0001

arob arob

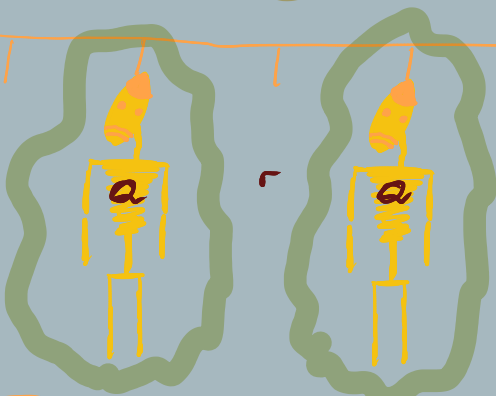
a występuje na pierwszej i 3 pozycji, więc droid a zostaje osłonięty



0010

ar abarob

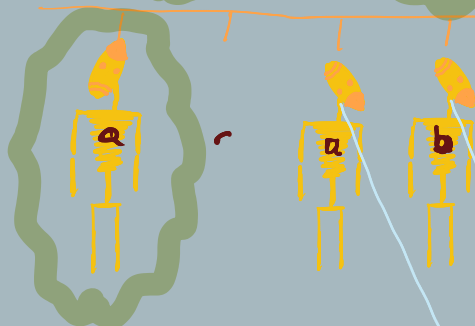
r występuje na 2, więc osłania a, ale siebie nie



0101

aa abarob

a osłania 1 i 3 pozycje



1000

arob abarob

b osłania 4 pozycje

Zauważmy, że gdybyśmy mieli wzorec arob, to litera a na 3 pozycji nie zostałaby odstrzelona, bo mogłaby być początkiem nowego wystąpienia wzorca.

Droid przetrwa  $\Leftrightarrow$  wzorec został dopasowany: -)

# Algorytm KMP (Knuth, Morris, Pratt)

Def: Funkcja  $\pi: \{1, \dots, m\} \rightarrow \{0, \dots, m-1\}$

$$\pi(q) = \max\{k : k < q \wedge P_k \supset P_q \wedge P_k \subset P_q\},$$

gdzie  $P_k$  oznacza słowo  $p_1 p_2 \dots p_k$  (bądź więcej  
tę oznaczenie  $p_1 \dots p_k$ )

Czyli chcemy mieć maksymalny prefiks właściwy (tj. nie będący równy słowu, którego jest prefiksem), który jest jednocześnie sufiksem (bądź więcej nazwy prefiksosufiks lub w skrócie pref-suf).

Przykład:

$P = ababac$

| nr litery | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|-----------|---|---|---|---|---|---|---|
| $\pi(i)$  | 0 | 0 | 1 | 2 | 3 | 0 | 1 |

$i=1$   
a, najwięk. pref-suf

$i=2$   
ab, najwięk. pref-suf

$i=3$

aba  
1

$i=6$

ababac  
0

$i=4$

abab  
2

$i=7$

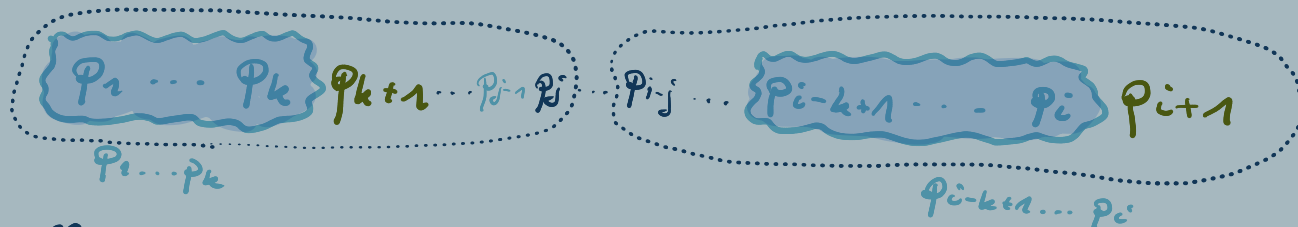
ababaca  
1

$i=5$

ababac  
3

Jak dynamicznie znaleźć  $\pi(i+1)$ ?

10



Niech  $P_{k+1} = P_{i+1}$  - wtedy największy prefiks-sufiks uzyskamy dodając literkę  $P_{k+1} = P_{i+1}$  do pref-suf dla  $P_1 \dots P_{i+1}$ . Gdyby istniał dłuższy (np.  $P_1 \dots j$ ) to implikuje to istnienie dłuższego dla  $P_1 \dots i$

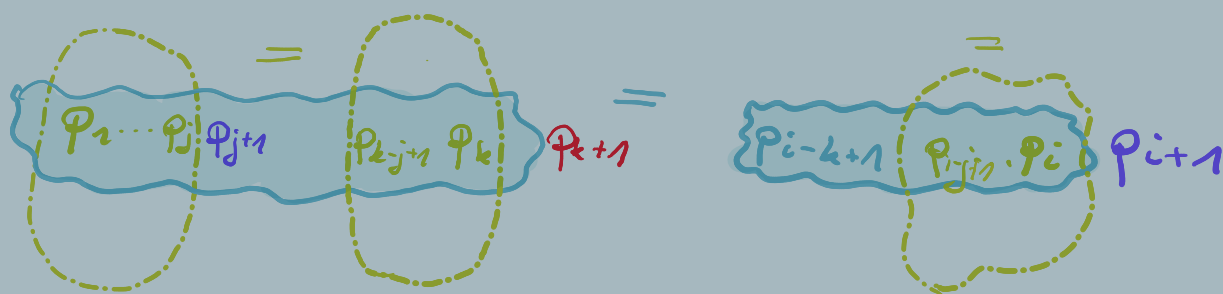
2°

$$p_1 \dots p_i p_{i+1}$$

Niech  $p_{i+1} \neq p_{k+1} \wedge k=0$ . Gdyby  $p_{i+1} = p_{k+1}$  moglibyśmy przedłużyć (przedy) prefiks dla  $p_1 \dots p_i$ , ale w tym wypadku nie możemy tego zrobić.

3°

$$p_1 \dots p_k = p_{i-k+1} \dots p_i \wedge p_{i-j+1} \dots p_i = p_1 \dots p_j \Rightarrow p_{k-j+1} \dots p_k = p_{i-j+1} \dots p_i$$



Niech  $p_{k+1} \neq p_{i+1}$ , ale  $k > 0$ . Założymy, że dla  $p_1 \dots p_{i+1}$  istnieje szukany prefiks. Niech będzie to  $p_1 \dots p_{j+1}$ , a def pref-suf  $p_{i-j+1} \dots p_i = p_1 \dots p_j$ , ale że  $p_1 \dots p_k$  jest pref-suf  $p_1 \dots p_i$  to  $p_1 \dots p_k = p_{i-k+1} \dots p_i$ , co oznacza, że  $p_{k-j+1} \dots p_k = p_{i-j+1} \dots p_i = p_1 \dots p_j$ , czyli że nasz największy pref-suf (poza ostatnią literkę) jest jednym z pref-suf  $p_1 \dots p_k$ , może nie największym dlatego może go poszukać w największym pref-suf  $p_1 \dots p_k$ .

Przykład

1° abacaba

$x(1) \neq 0$  bo nie istnieje właściwy prefiks do 1 literki

2° abacaba

$x(2) \neq 0$ , bo  $k=x(1)=0$  i  $a=p_{k+1}=p_2=b$

3° abacaba

$x(3) \neq 1$ , bo  $q_3 = p_{x(2)} = p_0 = a$

4° abacaba

$k \neq x(3) = 1$ , ale  $b = p_{k+1} = p_2 \neq p_4 = c$ , więc szukamy prefiksu w  $p_1 \dots p_{x(3)} = p_1$

abacaba

$x(1) = 0$ , więc (przypadek 2)  $x(4) \neq 0$

5° abacaba

$k \neq x(4) = 0$ , ale  $a = p_1 = p_{k+1} = p_5 = a$ , więc  $x(5) \neq x(4) + 1 = 1$

6° abacaba

$k \neq x(5) = 1$   $b = p_{k+1} = p_2 = p_6 = b$ , więc  $x(6) = x(5) + 1 = 2$

7° abacaba

$x(7) \neq x(6) + 1 = 3$

# Pseudokod:

```
 $\pi[1] \leftarrow 0$   
 $k \leftarrow 0$  // pozycja o tekście  
for  $q = 2 \dots m$   
    while  $(k > 0 \wedge P[k+1] \neq P[q])$   $k \leftarrow \pi(k)$   
    if  $(P[k+1] = P[q])$   $k \leftarrow k+1$   
     $\pi[q] \leftarrow k$   
return  $\pi$ 
```

Koszt  $\Theta(m)$       UWAGA -  $\pi$  można budować podczas wyszukiwania wzorca.

↳ Można to pokazać dzięki po 2 monetki na każdy m. Jeżeli while się nie odbył, to zoszczędzoną monetkę można reinwestować w następny index. Wtedy w q-tej iteracji mamy do dyspozycji  $\pi(q-1) + 1$  monetek, bo w każdym obrocie while w najgorszym przypadku zmniejszamy k tylko o 1, bo potrzymamy się na prekursorze prekursora, każde takie zmniejszenie kosztuje monetkę. Początkowo k jest równe  $\pi(q-1)$ , więc zawsze będzie co najmniej o 1 mniejsze od stanu konta. Więc jak znajdziemy odpowiednie  $\pi(k)$

mamy  $\pi(k) + 1 = \pi(q)$  monetek. Jedną poświęcimy na jakiś stały liczbę operacji (inkrementacja k, q, przejście do kolejnej iteracji), więc w kolejnym obrocie mamy  $\pi(q) - 1 + 2 = \pi(q) + 1$  monetek. Zatem wykonamy  $\alpha 2m$  operacji.

## Algorytm KMP - idea:

$P_1 \dots P_j$   $P_{j+1} \dots P_m$   
 $t_1 \dots t_{i-j} \dots t_{i-1}$   $t_i \dots t_n$

Łożymy, że dopasowaliśmy fragment wzorca, ale kolejna litera nie pasuje. Chcielibyśmy jakos' przesunąć wzorec.

$P_1 \dots P_{j-1} P_{j+1} \dots P_j P_{j+1} \dots P_m$   
 $P_1 \dots P_{j-1} P_{j+1} \dots P_j P_{j+1} \dots P_m$   
 $t_1 \dots t_{i-j} \dots t_{i-1} t_i \dots t_n$

Ale zauważmy, że jeśli da się ten wzorec przesunąć (np. o  $j$ !) to zachodzić musi  $P_1 = t_{i-j+1} = P_{j+1}, \dots, P_j = t_{i-1} = P_j$ , czyli przesuwamy do największego pref-suf (największego, bo nie chcemy zominąć żadnego dopasowania).

1. Wyznaczyć  $\pi$

2.  $q \leftarrow 0$  // ile znaków dopasowaliśmy do wzorca

for  $i \leftarrow 1 \dots n$

while  $q > 0 \wedge P[q+1] \neq T[i]$ :

$q \leftarrow \pi[q]$

if  $(P[q+1] = T[i])$

$q \leftarrow q + 1$

if  $(q = m)$

print (stdin, "wzorec jest z przeszczeniem %i", i - m)

$q \leftarrow \pi[q]$

Skomplikacja

Koszt obliczenia  $\pi$  + #zwiększenia  $q \leq n$  +

#zmniejszenia  $q \leq$  #zwiększeń

$\rightarrow O(n)$  (argument, że while nie może niekiedy być podobny jak przy obliczaniu  $\pi$ )

# Przykład

Wzorec     $aab$   
 Tekst     $aaab$

| $i$     | 1 | 2 | 3 |
|---------|---|---|---|
| $\pi_i$ | 0 | 1 | 0 |

$aaab$   
 $i$   
 $aab$   
 $q+1$



$aaab$   
 $i$   
 $aab$   
 $q+1$



$aaab$   
 $i$   
 $aab$   
 $q+1$

niepasuje -  
 przesuwamy wzorec  
 na pozycje  $\pi(2)=2$

sprawdzamy  
 pref-suf  $aa$

$\rightarrow$  dopasowaliśmy

różne z def  $\rightarrow$  ||  
 $aaab$   
 $q q+1$

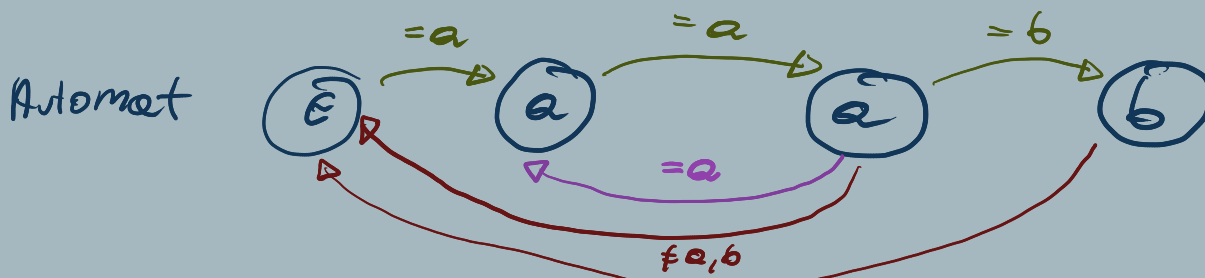
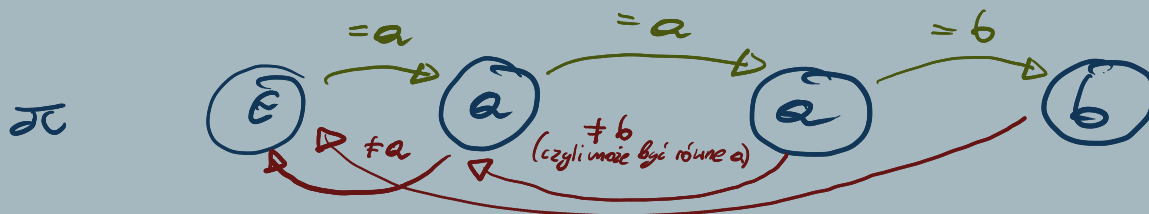


$aaab$   
 $q q+1$



$aaab$   
 $q q+1$

Różnica między  $\pi$  a poprzednią definicję automatu:



Ważna  $\pi$  - niepotrzebujemy ustaleć decyzji dla każdej litery  
 alfabetu.